EXECUTION PLAN

Phase 25 -> Phase 31

Evaluation - Guardrails - Auth - UI - MLOps - AWS - Monitoring

Concrete deliverables - file layouts - acceptance criteria - tools

From working prototype to deployed, hireable, top-1% system

Companion to System Design v2.0

How to Read This Plan

Each phase below is a self-contained milestone with the same structure: **Objective** (why it matters for a top-1% / hireable project), **Deliverables** (what must exist when done), **File & Folder Layout** (exactly where code goes), **Implementation Steps** (ordered work), **Tools** (open-source where possible, matching your stack), and **Acceptance Criteria / Definition of Done** (objective gates).

Phases are sequenced deliberately: you cannot meaningfully build CI quality gates (P29) before an eval harness exists (P25); you should not expose a public UI (P28) before guardrails (P26) and auth (P27). Follow the order unless a dependency note says otherwise.

Phase	Milestone	Primary Outcome	Depends On
25	Evaluation	Measurable RAG quality (recall@k, faithfulness)	P24
26	Guardrails	Safe I/O: jailbreak, PII egress, schema	P25
27	Authentication	Multi-user, JWT/OAuth, tenant isolation	P24
28	Gradio UI	Multimodal chat UX with streaming + sources	P26, P27
29	MLOps / CI-CD	Automated test+eval gates, model registry	P25
30	AWS Deployment	Containerized cloud deploy + IaC	P27, P29
31	Monitoring	Live SLOs, alerting, RAG-quality dashboards	P30

Estimated effort assumes one focused engineer using the .claude dev-team skills (File 3) to accelerate. These are working estimates, not hard commitments.

PHASE 25 - Evaluation Harness & RAG Quality Metrics

Estimated effort: 3-5 days | **Risk if skipped:** CRITICAL - without this you are shipping a system whose answer quality is unmeasured and can silently regress.

Objective

Make answer and retrieval quality **measurable and regression-gated**. This is the single highest-signal addition for a top-1% portfolio project: it proves you think like an ML engineer, not just an app developer. Interviewers at top startups specifically probe for this.

Deliverables

- **app/eval/** module with a reusable evaluation runner.
- A labeled gold dataset (40-100 query / ground-truth-context / reference-answer triples) drawn from your own corpus.
- Retrieval metrics: recall@k, MRR, nDCG, context precision.
- Generation metrics: faithfulness, answer relevance, context recall (via Ragas).
- Hallucination rate tracked explicitly (not assumed zero).
- A single CLI: `python -m app.eval.run --suite full` -> JSON + Markdown report.
- A non-zero exit code when any metric falls below a configured threshold (CI gate hook for P29).
- Baseline numbers committed so future changes are comparable.

File & Folder Layout

```
app/eval/
__init__.py
datasets/
gold_set.jsonl # {query, relevant_ids[], reference_answer}
build_gold_set.py # bootstrap from data/raw + manual review
metrics/
retrieval.py # recall@k, mrr, ndcg, context_precision
generation.py # ragas wrappers (faithfulness, answer_rel)
hallucination.py # ungrounded-claim detector
runner.py # orchestrates: load -> run pipeline -> score
thresholds.yaml # min recall@5, min faithfulness, etc.
report.py # writes rag_report.{json,md}
run.py # CLI entrypoint (python -m app.eval.run)
tests/eval/test_eval_smoke.py
```

Implementation Steps

- 1. Build the gold set: sample real queries your users would ask against the existing corpus in data/raw; for each, record the truly-relevant chunk IDs and a human reference answer. Keep it small but honest.
- 2. Implement retrieval metrics against the hybrid retriever output (BM25 + Qdrant + reranker) so you measure the real pipeline, not a toy.
- 3. Wire Ragas for generation metrics; it integrates with LangChain objects and works with your local GGUF model as the judge (or a small open judge model).
- 4. Add a hallucination check: claims in the answer not entailed by retrieved context are flagged.
- 5. Externalize thresholds to thresholds.yaml; runner exits non-zero on breach.
- 6. Commit baseline rag_report.json so regressions are visible in diffs.

Tools

Concern	Tool	Why
RAG metrics	Ragas	Purpose-built, LangChain-native, open-source
Retrieval math	NumPy / scikit-learn	recall@k, nDCG, MRR
Judge model	Local GGUF Mistral or small open judge	Keeps it $0 + offline
Experiment tracking (opt.)	MLflow	Versioned eval runs for P29

Acceptance Criteria (Definition of Done)

- `python -m app.eval.run --suite full` runs end-to-end and writes both reports.
- Baseline metrics are committed and documented in README.
- Lowering retrieval k or breaking the reranker causes the run to FAIL (exit != 0).
- Every metric has a documented threshold and rationale.

PHASE 26 - Guardrails (Input & Output Safety)

Estimated effort: 3-4 days | **Depends on:** P25 (so you can measure that guardrails don't degrade quality).

Objective

You already have *input-side* injection sanitization scattered across controller, router, embedder, captioner, and web_search. Phase 26 **consolidates** these into one auditable layer and adds the missing **output-side** guardrails: jailbreak detection, PII egress prevention, toxicity, and schema-constrained answers. This is what makes the system safe to expose publicly in P28.

Deliverables

- **app/guardrails/** single source of truth for all I/O safety.
- Input guard: unify the existing scattered injection patterns + length + language + modality checks.
- Output guard: groundedness check (reuse P25 hallucination), PII egress scrub (Presidio, already a dependency), toxicity filter, and refusal handling.
- Policy config so thresholds/patterns are data, not code.
- Audit log: every block/allow decision recorded with reason + correlation ID.
- Guardrail metrics exported to Prometheus (blocks by type).

File & Folder Layout

```
app/guardrails/
__init__.py
input_guard.py # consolidates controller/router/embedder logic
output_guard.py # groundedness, PII egress, toxicity, schema
jailbreak.py # classifier (open model or rule+embedding)
policies.yaml # patterns, thresholds, refusal templates
audit.py # structured, signed decision log
exceptions.py # GuardrailBlocked(reason, surface)
tests/guardrails/ (red-team prompt suite: 50+ adversarial cases)
```

Implementation Steps

- 1. Inventory every existing _sanitize / _INJECTION_PATTERNS / _is_ssrf_risk site (controller, router, text_embedder, frame_captioner, web_search) and move the logic into input_guard.py; call the unified guard from those sites instead of duplicating.
- 2. Build output_guard: before returning, verify the answer is grounded in retrieved context (reuse P25), scrub residual PII, run a toxicity check, and enforce the AgentResponse schema.
- 3. Add a jailbreak detector: start with rules + embedding similarity to known jailbreaks; upgrade to a small open classifier if needed.
- 4. Centralize all patterns/thresholds in policies.yaml.
- 5. Write a red-team test suite (prompt injection, data exfil, PII bait, jailbreaks) and assert blocks. Use P25 to confirm guardrails don't tank quality on benign queries.

Tools

Concern	Tool
PII detect/anonymize	Microsoft Presidio (already used in memory layer)
Guardrail framework (opt.)	NeMo Guardrails or Guardrails-AI (open-source)
Toxicity	Detoxify / open classifier

Concern	Tool
Jailbreak signal	Rules + embedding similarity; small open clf
Schema enforcement	Pydantic v2 (already in stack)

Acceptance Criteria

- All injection logic flows through app/guardrails (no duplicated pattern lists).
- Red-team suite: 100% of adversarial cases blocked or safely refused.
- P25 quality metrics do NOT regress beyond a small tolerance with guardrails on.
- Every decision is in the audit log with a reason.

PHASE 27 - Authentication & Multi-User Isolation

Estimated effort: 4-6 days | **Depends on:** P24. Can run in parallel with P25/26.

Objective

Turn a single-user prototype into a real **multi-tenant** service. The highest-value part is not the login screen - it is **data isolation**: user A must never retrieve user B's documents, memory, or summaries. Your infra already hints at this (gdpr_purge takes user_id/session_id; Qdrant payload filters); Phase 27 makes it enforced and authenticated.

Deliverables

- **app/auth/** with registration, login, JWT issuance + refresh, password hashing.
- OAuth2 (Google/GitHub) optional social login.
- FastAPI dependency that injects the authenticated user into every request.
- Per-user tenant scoping enforced at Qdrant (payload filter user_id), Redis (key namespace), Mongo (document filter).
- Rate limiting per user; role support (user/admin).
- User-facing GDPR endpoint wired to existing infra_registry.gdpr_purge.

File & Folder Layout

```
app/auth/
__init__.py
models.py # User, Role, Token (Pydantic + DB schema)
service.py # register, authenticate, hash (argon2/bcrypt)
jwt_handler.py # issue/verify/refresh access tokens
oauth.py # optional Google/GitHub OAuth2
dependencies.py # get_current_user() FastAPI dependency
tenancy.py # build per-user Qdrant/Redis/Mongo scopes
rate_limit.py # per-user limiter
app/api/middleware.py # auth middleware + request user context
tests/auth/ (authz tests: cross-tenant access MUST fail)
```

Implementation Steps

- 1. Define User/Role models; store users in Mongo Atlas (already provisioned).
- 2. Implement Argon2/bcrypt password hashing and JWT access+refresh tokens.
- 3. Add get_current_user() dependency; protect all query/ingest routes.
- 4. tenancy.py: every retrieval adds a Qdrant payload filter `user_id == current`; Redis keys prefixed `u:{id}:`; Mongo queries filtered by user_id. Backfill ingestion to stamp user_id into payloads.
- 5. Add per-user rate limiting and an admin role.
- 6. Expose POST /me/forget -> infra_registry.gdpr_purge(user_id=...).
- 7. Write authz tests proving user A CANNOT read user B data through any path (retrieval, memory, summary).

Tools

Concern	Tool
Auth tokens	python-jose / PyJWT (JWT)
Password hashing	passlib[argon2] or bcrypt
OAuth2	Authlib (open-source)
User store	MongoDB Atlas (already in stack)
Rate limiting	slowapi / custom Redis token bucket

Acceptance Criteria

- All protected routes reject missing/invalid tokens (401).
- Cross-tenant test: user A querying returns ZERO of user B's chunks/memory.
- GDPR endpoint provably purges across Qdrant + Redis + Mongo.
- Tokens expire and refresh correctly; passwords never stored in plaintext.

PHASE 28 - Gradio UI (Multimodal Chat)

Estimated effort: 3-4 days | **Depends on:** P26 (safe outputs) + P27 (auth). This is the demo recruiters actually click.

Objective

A polished, multimodal chat interface that showcases the system's strengths: file upload across all 7 modalities, streamed answers, visible routing decision (RAG vs Web vs Memory vs Direct), and cited sources. The UI is your portfolio's first impression - it must feel production-grade, not a notebook.

Deliverables

- **ui/gradio_app.py** - chat UI with streaming responses.
- Multimodal upload: text, pdf, word, excel, image, audio, video.
- Login screen wired to P27 auth; per-session conversation memory.
- Transparency panel: shows chosen route, confidence, retrieved sources, latency.
- Source citations with snippets; thumbs up/down feedback captured to a store (feeds P25 dataset growth).
- Graceful error + loading states; mobile-reasonable layout.

File & Folder Layout

```
ui/
gradio_app.py # main Blocks app, launch()
components/
chat.py # streaming chat panel
uploader.py # multimodal file intake
transparency.py # route/confidence/sources/latency panel
auth_panel.py # login/register (calls app.auth)
client.py # thin async client -> FastAPI backend
theme.py # consistent styling
feedback.py # store thumbs up/down -> eval dataset
```

Implementation Steps

- 1. Build a thin client that calls your existing FastAPI routes (do NOT bypass the API - the UI must use the same hardened path).
- 2. Implement streaming chat using the GGUF model's token stream.
- 3. Add multimodal uploader that posts to the ingestion endpoint and shows progress.
- 4. Add the transparency panel surfacing AgentDecision (route, reason, confidence) + sources + latency already returned in your AgentResponse.
- 5. Gate the app behind P27 login; bind conversation to session_id.
- 6. Capture feedback and append to app/eval/datasets to grow the gold set over time.

Tools

Concern	Tool
UI framework	Gradio (Blocks API), open-source
Backend calls	httpx async client -> FastAPI
Streaming	Gradio streaming + GGUF token stream
Feedback store	Mongo Atlas (reuse) or JSONL

Acceptance Criteria

- All 7 modalities can be uploaded and queried from the UI.
- Answers stream token-by-token; sources and route are visible.
- UI requires login; sessions are isolated per user.
- Feedback is persisted and consumable by the P25 harness.

PHASE 29 - MLOps / LLMOps / CI-CD

Estimated effort: 4-6 days | **Depends on:** P25 (the eval gate has nothing to enforce without it).

Objective

Automate quality. Every pull request must pass lint, type-check, unit + integration tests, AND the P25 RAG-quality gate before it can merge. Models and indexes must be versioned and reproducible. This is the phase that signals senior-level engineering maturity to hiring managers.

Deliverables

- Containerization: multi-stage Dockerfile (non-root, slim) + docker-compose for local full-stack.
- GitHub Actions: ci.yml (lint+type+test), eval-gate.yml (P25 thresholds block merge), cd.yml (build+push image on tag).
- Pre-commit hooks: ruff, black, mypy, detect-secrets.
- Model/version registry: pinned GGUF hash, embedding model+dim, Qdrant collection schema recorded and asserted at startup.
- Experiment tracking with MLflow for eval runs over time.
- Makefile / task runner for one-command local ops.

File & Folder Layout

```
.github/workflows/
ci.yml # ruff + mypy + pytest (matrix py3.11)
eval-gate.yml # runs app.eval.run; fails PR on regression
cd.yml # on tag: docker build -> push to registry
deploy/
Dockerfile # multi-stage, non-root
docker-compose.yml # api + qdrant(local) + redis + mongo
.dockerignore
.pre-commit-config.yaml
Makefile
app/core/version_registry.py # asserts model hash + dims at boot
```

Implementation Steps

- 1. Write a multi-stage Dockerfile: builder installs deps + downloads GGUF (checksum-verified via your existing download_model.py); runtime is slim, non-root.
- 2. docker-compose for a full local stack (API + local Qdrant/Redis/Mongo) so contributors run everything with one command.
- 3. ci.yml: ruff + mypy + pytest with coverage gate (>=80% on core).
- 4. eval-gate.yml: spins minimal services, runs `python -m app.eval.run`, fails the PR if thresholds.yaml is breached - the regression gate.
- 5. cd.yml: on git tag, build and push the image to a registry (GHCR/ECR).
- 6. version_registry.py asserts the running GGUF hash + embedding dim + Qdrant schema match config at startup; refuse to boot on mismatch.
- 7. Add pre-commit + Makefile (make test, make eval, make run, make docker).

Tools

Concern	Tool
CI/CD	GitHub Actions (free for public repos)
Containers	Docker, docker-compose

Concern	Tool
Lint/format/type	ruff, black, mypy
Secret scanning	detect-secrets / gitleaks
Experiment tracking	MLflow (open-source)
Registry	GHCR or AWS ECR

Acceptance Criteria

- A PR that lowers retrieval quality is automatically BLOCKED by eval-gate.yml.
- `docker compose up` brings the entire system online locally.
- CI is green on main; coverage gate enforced.
- Startup refuses to boot on model/dimension/schema mismatch.

PHASE 30 - AWS Deployment

Estimated effort: 5-8 days | **Depends on:** P27 + P29. A live URL is the difference between 'project' and 'shipped product' on a CV.

Objective

Deploy the containerized system to AWS with infrastructure-as-code so it is reproducible, and a public HTTPS endpoint a recruiter can actually visit. Keep cost near-zero using free tier + the fact that your LLM is local (no GPU bill, no API bill).

Deliverables

- IaC (Terraform) describing all AWS resources - no click-ops.
- Container deployment: ECS Fargate (recommended) or EC2 + compose; CPU-only sizing for the GGUF model.
- Secrets in AWS Secrets Manager / SSM (Qdrant/Redis/Mongo/Tavily keys) - never in image.
- HTTPS via ALB + ACM certificate; domain optional.
- Persistent model storage (EFS or baked image layer) so the 4GB GGUF isn't re-downloaded.
- Connectivity to existing managed clouds (Qdrant/Upstash/Atlas) over TLS.
- Runbook: deploy, rollback, scale, incident.

File & Folder Layout

```
deploy/aws/
terraform/
main.tf vpc.tf ecs.tf alb.tf secrets.tf iam.tf
variables.tf outputs.tf backend.tf
task-definition.json # ECS Fargate task (CPU sized)
buildspec.yml # optional CodeBuild
deploy.sh rollback.sh
docs/runbook.md # deploy/rollback/scale/incident
```

Implementation Steps

- 1. Terraform: VPC, subnets, ECS cluster, ALB + ACM cert, ECR repo, IAM roles, Secrets Manager entries.
- 2. Size the Fargate task for CPU GGUF inference (generous RAM, no GPU); tune llama.cpp threads via env.
- 3. Store the GGUF on EFS (or bake into the image) so cold starts don't re-download 4GB.
- 4. Inject cloud creds from Secrets Manager at runtime; confirm TLS to Qdrant/Upstash/Atlas.
- 5. Wire cd.yml (P29) to push image to ECR and update the ECS service.
- 6. Smoke-test the public HTTPS endpoint end-to-end (upload + query + auth).
- 7. Write runbook.md with exact deploy/rollback/scale/incident commands.

Tools

Concern	Tool
IaC	Terraform (open-source)
Compute	AWS ECS Fargate (CPU) or EC2
Registry	AWS ECR
Secrets	AWS Secrets Manager / SSM Parameter Store
TLS/Ingress	Application Load Balancer + ACM

Concern	Tool
Model storage	EFS or image layer

Acceptance Criteria

- `terraform apply` from zero produces a working public HTTPS endpoint.
- No secret is present in the image or repo; all from Secrets Manager.
- Cold start does not re-download the 4GB model.
- Documented rollback works and is tested once.

PHASE 31 - Monitoring & Observability (Live)

Estimated effort: 3-5 days | **Depends on:** P30. Closes the loop: you already emit Prometheus + OTel in code - now make it visible and alert on it.

Objective

Turn the telemetry your code *already produces* (Prometheus metrics, OTel spans, structured logs) into live dashboards, SLOs, and alerts - including **RAG-quality SLOs**, not just CPU/latency. Demonstrating production monitoring of model quality is genuinely top-1%.

Deliverables

- Prometheus deployment scraping the app's /metrics.
- Grafana dashboards: system (latency, errors, circuit breakers) + RAG quality (recall@k, faithfulness trend, route distribution, hallucination rate).
- OpenTelemetry collector exporting traces to a backend (Tempo/Jaeger).
- Alerting: error spike, circuit breaker open, p95 latency breach, eval-metric drift.
- Scheduled online eval job re-running P25 on sampled live traffic.
- SLO doc: target numbers + error budgets.

File & Folder Layout

```
monitoring/
prometheus/prometheus.yml # scrape config
grafana/dashboards/
system_health.json
rag_quality.json # recall@k, faithfulness, routes
otel/collector-config.yaml
alerts/rules.yml # alerting rules
slo.md # SLOs + error budgets
jobs/online_eval.py # scheduled sampled re-eval
```

Implementation Steps

- 1. Deploy Prometheus (compose/ECS) scraping the existing /metrics endpoint.
- 2. Build Grafana dashboards: one for system health, one for RAG quality fed by P25 metrics pushed as gauges.
- 3. Stand up an OTel collector; route the spans your code already emits to Tempo/Jaeger for trace inspection of any bad answer.
- 4. Define alert rules (error rate, breaker open, p95 latency, quality drift) -> email/Slack webhook.
- 5. Schedule online_eval.py to sample real traffic and re-run the P25 suite, pushing results to the RAG-quality dashboard.
- 6. Write slo.md with concrete targets and error budgets.

Tools

Concern	Tool
Metrics	Prometheus (already instrumented in code)
Dashboards	Grafana (open-source)
Tracing	OpenTelemetry Collector + Tempo/Jaeger
Logs	structlog JSON -> Loki (optional)
Alerting	Alertmanager / Grafana alerts

Acceptance Criteria

- A bad answer can be traced end-to-end from the dashboard to its spans.
- RAG-quality dashboard shows recall@k + faithfulness trends over time.
- Alerts fire on injected failure (test by forcing a breaker open).
- SLOs documented with error budgets.

Cross-Phase: What Makes This 'Top 1% / Hireable'

Most candidate RAG projects stop at 'it answers questions'. The phases above deliberately add the four things that separate a top-1% portfolio project from the other 99%:

Signal	Where It Comes From	Why Recruiters Care
Measured quality	P25 eval harness + baselines	Proves ML-engineer rigor, not just app glue
Safety under adversaries	P26 guardrails + red-team suite	Production trust; security awareness
Real multi-tenancy	P27 auth + provable isolation	Systems thinking; security correctness
Shipped + observed	P29-31 CI/CD, AWS, monitoring	You can deliver, not just prototype

Sequencing reminder: P25 before P29 (gate needs metrics); P26+P27 before P28 (don't expose unsafe/unauthed UI); P29 before P30 (deploy what CI blesses); P30 before P31 (monitor what's live).

Next: File 3 delivers the .claude dev-team skills that execute this exact plan inside Claude Code - an orchestrator/tech-lead plus specialist engineers (architect, code-review, security, perf, eval, guardrails, devops) that read this plan and drive each phase to its Definition of Done.